

Lab 2: Leslie Matrices & Population Dynamics

CAS CS 132: Geometric Algorithms

Teaching Fellow: Angelos Poulis (apoulis@bu.edu)

Feb. 9th 2026

Plotting you will need for this lab

Using Matplotlib Pyplot

In this lab, you will use Matplotlib to:

- Plot population over time
- Compare multiple scenarios on the same graph
- Add labels, title, legend
- Control axis limits

Generating a plot in Matplotlib

Template

In general, you follow these steps:

1. **Imports**
2. Prepare x and y data
3. Plot the data
4. Add labels and title
5. Show the plot

We will use *line* plots throughout the lab.

```
import matplotlib.pyplot as plt
import numpy as np # to generate example data

x = np.arange(10)
y = x**2

plt.plot(x, y)

plt.xlabel("time")
plt.ylabel("population")
plt.title("Example line plot")

plt.show()
```

Generating a plot in Matplotlib

Template

In general, you follow these steps:

1. Imports
- 2. Prepare x and y data**
3. Plot the data
4. Add labels and title
5. Show the plot

We will use *line* plots throughout the lab.

```
import matplotlib.pyplot as plt
import numpy as np # to generate example data

x = np.arange(10)
y = x**2

plt.plot(x, y)

plt.xlabel("time")
plt.ylabel("population")
plt.title("Example line plot")

plt.show()
```

Generating a plot in Matplotlib

Template

In general, you follow these steps:

1. Imports
2. Prepare x and y data
- 3. Plot the data**
4. Add labels and title
5. Show the plot

We will use *line* plots throughout the lab.

```
import matplotlib.pyplot as plt
import numpy as np # to generate example data

x = np.arange(10)
y = x**2

plt.plot(x, y)

plt.xlabel("time")
plt.ylabel("population")
plt.title("Example line plot")

plt.show()
```

Generating a plot in Matplotlib

Template

In general, you follow these steps:

1. Imports
2. Prepare x and y data
3. Plot the data
- 4. Add labels and title**
5. Show the plot

We will use *line* plots throughout the lab.

```
import matplotlib.pyplot as plt
import numpy as np # to generate example data

x = np.arange(10)
y = x**2

plt.plot(x, y)

plt.xlabel("time")
plt.ylabel("population")
plt.title("Example line plot")

plt.show()
```

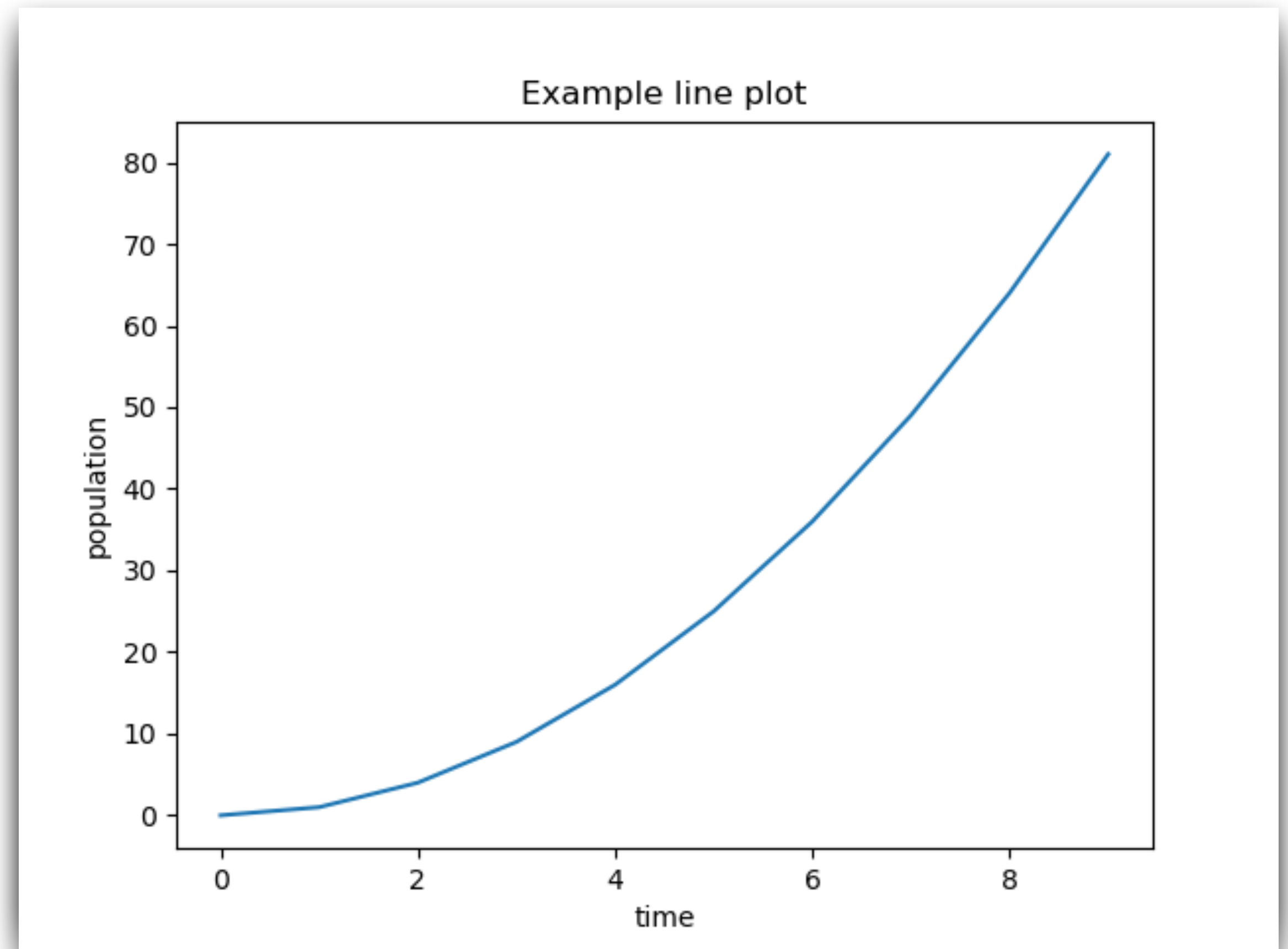
Generating a plot in Matplotlib

Template

In general, you follow these steps:

1. Imports
2. Prepare x and y data
3. Plot the data
4. Add labels and title
- 5. Show the plot**

We will use *line* plots throughout the lab.



Generating a plot in Matplotlib

Formatting

- **Add multiple lines on the same plot.**
- Change the style (here, color) of each line. You can also change linestyle (e.g., dashed), or use markers, and so on...
- Fix axis limits.
- Plot the legend which maps each line to its label!

```
plt.figure() # Create a new figure

# Multiple population lines
plt.plot(x, y1, color='r', label="control")
plt.plot(x, y2, color='g', label="s[4] = 1")
plt.plot(x, y3, color='b', label="s[3] = 1")

plt.xlabel("time")          # Label for x-axis
plt.ylabel("population")    # Label for y-axis
plt.title("Population dynamics")

# Axis limits
plt.xlim(0, X_LIMIT) # Set x-axis limits
plt.ylim(0, Y_LIMIT) # Set y-axis limits

plt.legend() # Maps labels to lines

plt.show()
```

Generating a plot in Matplotlib

Formatting

- Add multiple lines on the same plot.
- **Change the style (here, color) of each line. You can also change linestyle (e.g., dashed), or use markers, and so on...**
- Fix axis limits.
- Plot the legend which maps each line to its label!

```
plt.figure() # Create a new figure

# Multiple population lines
plt.plot(x, y1, color='r', label="control")
plt.plot(x, y2, color='g', label="s[4] = 1")
plt.plot(x, y3, color='b', label="s[3] = 1")

plt.xlabel("time") # Label for x-axis
plt.ylabel("population") # Label for y-axis
plt.title("Population dynamics")

# Axis limits
plt.xlim(0, X_LIMIT) # Set x-axis limits
plt.ylim(0, Y_LIMIT) # Set y-axis limits

plt.legend() # Maps labels to lines

plt.show()
```

Generating a plot in Matplotlib

Formatting

- Add multiple lines on the same plot.
- Change the style (here, color) of each line. You can also change linestyle (e.g., dashed), or use markers, and so on...
- **Fix axis limits.**
- Plot the legend which maps each line to its label!

```
plt.figure() # Create a new figure

# Multiple population lines
plt.plot(x, y1, color='r', label="control")
plt.plot(x, y2, color='g', label="s[4] = 1")
plt.plot(x, y3, color='b', label="s[3] = 1")

plt.xlabel("time")          # Label for x-axis
plt.ylabel("population")    # Label for y-axis
plt.title("Population dynamics")

# Axis limits
plt.xlim(0, X_LIMIT) # Set x-axis limits
plt.ylim(0, Y_LIMIT) # Set y-axis limits

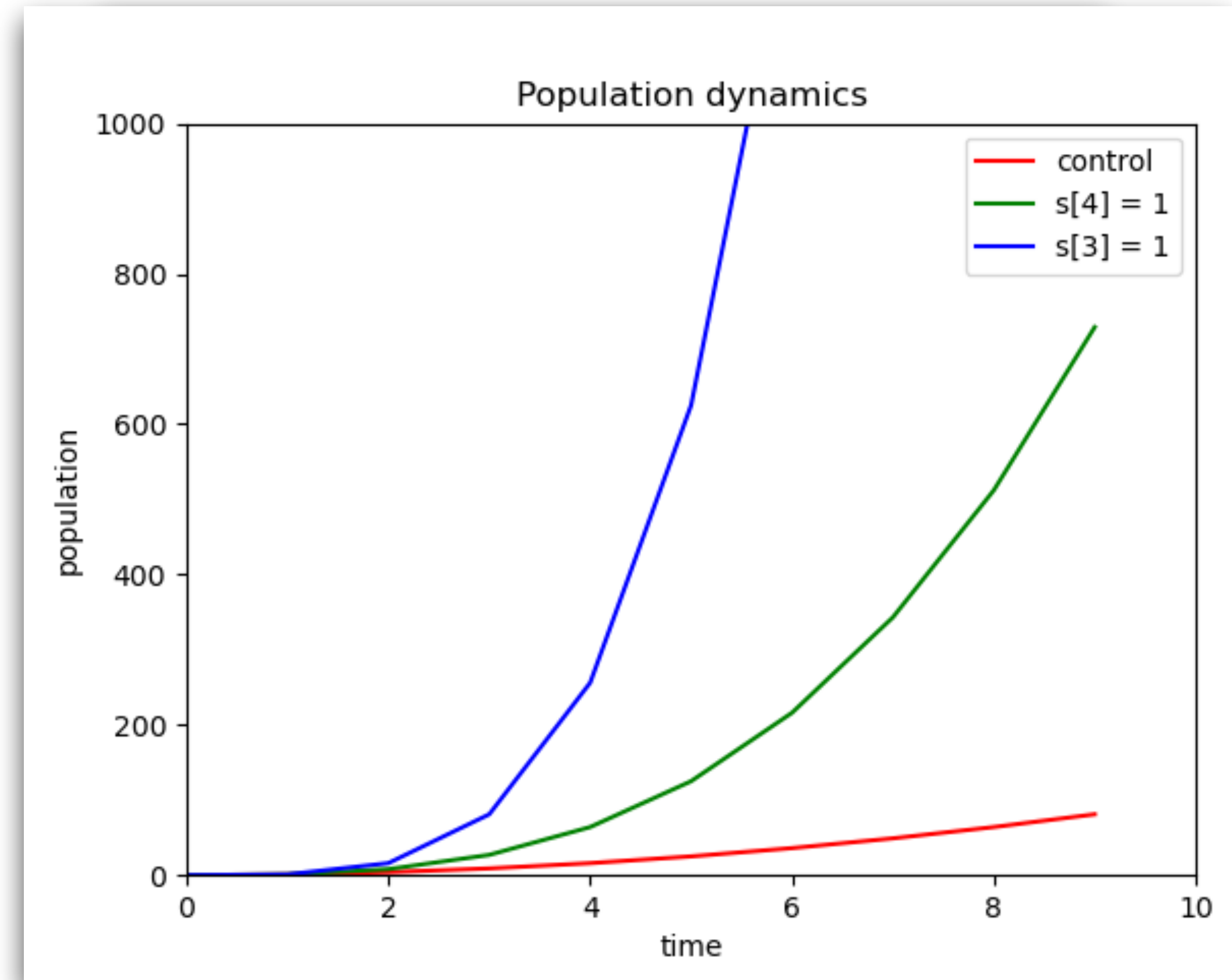
plt.legend() # Maps labels to lines

plt.show()
```

Generating a plot in Matplotlib

Formatting

- Add multiple lines on the same plot.
- Change the style (here, color) of each line. You can also change linestyle (e.g., dashed), or use markers, and so on...
- Fix axis limits.
- **Plot the legend which maps each line to its label!**



For more, visit: <https://matplotlib.org/stable/tutorials/pyplot.html#sphx-glr-tutorials-pyplot-py>

Population Dynamics

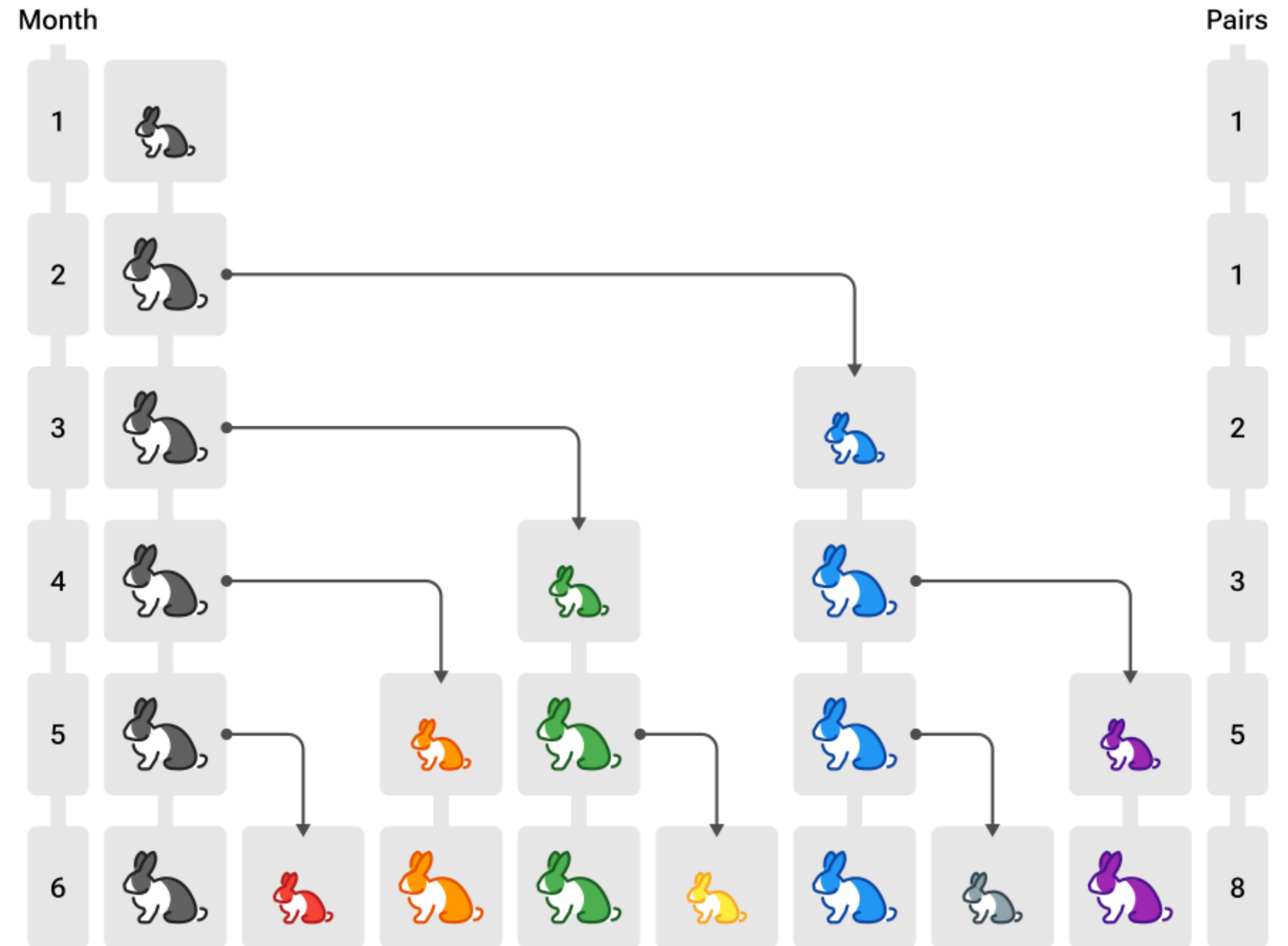
Using Linear Algebra

- Many real-world systems evolve over time (e.g., animal populations, demographics, etc)
- How does today's population determine tomorrow's population?
- **In this lab, we model population growth of rabbits using linear algebra.**

Fibonacci Sequence

For the rabbit problem

- The Fibonacci sequence:
$$F_n = F_{n-1} + F_{n-2}$$
- Originally came from a rabbit population model!¹
 - rabbits start as juveniles, then (after 1 month) become adults
 - adults reproduce once per month
 - no rabbits die (unrealistic but we will fix that later!)



¹<https://library.ethz.ch/en/locations-and-media/platforms/virtual-exhibitions/fibonacci-un-ponte-sul-mediterraneo/fibonaccis-significance-for-the-present-day/the-rabbit-problem.html>

Leslie matrices

Generalizing Fibonacci model

- Fibonacci's model is unrealistic: no deaths, only two age groups (adults, juveniles), fixed reproduction.
- Leslie matrices generalize this idea with:
 - multiple age groups
 - age-dependent survival
 - age-dependent reproduction rates

Modeling Example

As Matrix-Vector Multiplication [Fibonacci]

- Computing Fibonacci numbers is equivalent to *repeated* matrix-vector multiplication.

$$\begin{bmatrix} F_n \\ F_{n+1} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \begin{bmatrix} F_0 \\ F_1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}^n \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

- In the rabbit model:
 - F_n = number of adult rabbit pairs after n months
 - F_{n-1} = number of juvenile rabbit pairs

Modeling Example

As Matrix-Vector Multiplication [Leslie 2x2 Matrix case]

- We can generalize the Fibonacci sequence using a Leslie matrix, and a population vector.

$$\begin{bmatrix} J_n \\ A_n \end{bmatrix} = \begin{bmatrix} s_0 m_1 & s_0 m_2 \\ s_1 & s_2 \end{bmatrix} \begin{bmatrix} J_{n-1} \\ A_{n-1} \end{bmatrix}$$

J_n = population of juveniles at month n

A_n = population of adults at month n

s_i = survival probability of rabbit of age i

m_i = maternity rate of rabbit of age i

- **Link to Lab 2:** <https://nmmull.github.io/cs132-s26/labs/2/lab2.pdf>
- **Write-up due:** Feb 19, 2025 by 8:00PM.

